


UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

# EIN082B
# Taller de Lenguaje de Programación

Nancy Bernal Sánchez
Nancy.bernal@usm.cl




UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

# Explotación de Bases de Datos

Unidad N° 2



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

# Django: ORM, CRUD y Forms

Del modelo a la base de datos, y de la base de datos al formulario

# CONTENIDOS

01

## ORM

Modelos, migraciones y administrador de Django

02

## ORM v/s SQL

Consultas SQL directas con raw()

03

## CRUD con ORM

Listar, agregar, eliminar y modificar alumnos

04

## Clase Forms

Formularios a partir del modelo con ModelForm

05

## CRUD con Forms

El mismo CRUD para géneros, con menos código

# ¿QUÉ ES ORM?

**ORM (Object-Relational Mapping)** es una capa que traduce objetos de Python a tablas y sentencias SQL.

No escribimos el script de la base de datos: definimos una clase en `models.py` y Django crea la tabla.




**Ejemplo: la misma consulta en las dos formas**

ORM (Python)

```
Alumno.objects.all()
```

SQL

```
SELECT * FROM alumnos_alumno;
```

# SQL V/S ORM: TIPOS DE DATOS

Cada columna del script SQL tiene su equivalente como campo de la clase.

script.sql

```sql
CREATE TABLE alumno (
  rut         VARCHAR(10) PRIMARY KEY,
  nombre      VARCHAR(20),
  fechaNac    DATE,
  id_genero   INT,
  mail        VARCHAR(30),
  FOREIGN KEY (id_genero)
    REFERENCES genero(id)
    ON DELETE CASCADE
);
```

| SQL | Django (models.py) |
|---|---|
| VARCHAR(10) | models.CharField(max_length=10) |
| DATE | models.DateField() |
| INT | models.IntegerField() |
| INT AUTO_INCREMENT | models.AutoField(primary_key=True) |
| VARCHAR (correo) | models.EmailField() |
| FOREIGN KEY ... CASCADE | models.ForeignKey('Genero', on_delete=models.CASCADE) |
| PRIMARY KEY | primary_key=True |

# MODELOS DE EJEMPLO EN MODELS.PY

alumnos/models.py

```python
from django.db import models

class Genero(models.Model):
    id_genero = models.AutoField(db_column='idGenero', primary_key=True)
    genero    = models.CharField(max_length=20)

    def __str__(self):
        return str(self.genero)

class Alumno(models.Model):
    rut              = models.CharField(primary_key=True, max_length=10)
    nombre           = models.CharField(max_length=20)
    apellido_paterno = models.CharField(max_length=20)
    apellido_materno = models.CharField(max_length=20)
    fecha_nacimiento = models.DateField()
    id_genero        = models.ForeignKey('Genero', on_delete=models.CASCADE,
                                         db_column='idGenero')
    telefono         = models.CharField(max_length=45)
    email            = models.EmailField(unique=True, max_length=100,
                                         blank=True, null=True)
    direccion        = models.CharField(max_length=50, blank=True, null=True)
    activo           = models.IntegerField()

    def __str__(self):
        return str(self.nombre) + " " + str(self.apellido_paterno)
```

**`primary_key=True`**
define la clave primaria (rut, id_genero).

**`ForeignKey`**
relaciona cada alumno con un Genero.

**`blank / null`**
permiten dejar el campo vacío.

**`__str__`**
indica cómo se muestra el objeto, por ejemplo en el admin.

# MIGRACIONES Y ADMINISTRADOR DE DJANGO

## 1 Crear las tablas

```
python manage.py makemigrations
python manage.py migrate
```

Primero se crean las migraciones y luego se migra.

## 2 Crear súper usuario

```
python manage.py createsuperuser
```

Se pide usuario, correo y contraseña.

## 3 Registrar modelos

```
from .models import Genero, Alumno
admin.site.register(Genero)
admin.site.register(Alumno)
```

En admin.py, para que el administrador muestre las tablas.

## 4 Ingresar al admin

```
python manage.py runserver
http://127.0.0.1:8000/admin/
```

Agregar géneros (Masculino, Femenino, No binario) y 5 alumnos.

# CONSULTAR DATOS CON ORM

alumnos/views.py

```python
from django.shortcuts import render
from .models import Alumno, Genero

def index(request):
    alumnos = Alumno.objects.all()        # SELECT * FROM ...
    context = {"alumnos": alumnos}
    return render(request, 'alumnos/index.html', context)
```

templates/alumnos/index.html

```
<h1>Hola Alumnos</h1>
{% for alumno in alumnos %}
    {{ alumno.rut }}, {{ alumno.nombre }},
    {{ alumno.apellido_paterno }},
    {{ alumno.id_genero.genero }} <br>
{% endfor %}
```

## Etiquetas y variables

**{% comando %}**
Ejecuta lógica en el HTML: for, if, url.

**{{ objeto }}**
Muestra un valor, incluso de una tabla relacionada: alumno.id_genero.genero

**Probar en:**
`127.0.0.1:8000/alumnos/index`

# CONSULTAS SQL DIRECTAS CON RAW()

alumnos/urls.py

```
path('listadoSQL', views.listadoSQL, name='listadoSQL'),
```

alumnos/views.py

```
def listadoSQL(request):
    alumnos = Alumno.objects.raw('SELECT * FROM alumnos_alumno')
    context = {"alumnos": alumnos}
    return render(request, 'alumnos/listadoSQL.html', context)
```

**¿Por qué alumnos_alumno?**

`alumnos_alumno`

**alumnos** = nombre de la app
**alumno** = nombre del modelo, en minúsculas

👉 **Mismo resultado que con ORM:** /alumnos/listadoSQL muestra la misma lista que /alumnos/index, pero creada con SQL directo.

Para ver las tablas que crea Django se puede usar un gestor como DbVisualizer (dbvis.com).

# ¿QUÉ ES UN CRUD?

**CRUD** = Create, Read, Update, Delete. En español se le llama **Mantenedor**: la interfaz para agregar, buscar, modificar y eliminar datos.



## Create

Agregar

```
Alumno.objects.create(...)
```



## Read

Buscar / listar

```
Alumno.objects.all()
Alumno.objects.get(rut=pk)
```



## Update

Modificar

```
alumno.save()
```



## Delete

Eliminar

```
alumno.delete()
```

# FLUJO DE UNA ACCIÓN DEL CRUD

**Template**

Botón o enlace
{% url 'crud' %}

**urls.py**

path(...) asocia la
ruta con la vista

**views.py**

La función usa el
ORM (objects...)

**Template**

render(...) muestra
el resultado

### Rutas del CRUD de alumnos

alumnos/urls.py

```
urlpatterns = [
    path('crud', views.crud, name='crud'),
    path('alumnosAdd', views.alumnosAdd, name='alumnosAdd'),
    path('alumnos_del/<str:pk>', views.alumnos_del, name='alumnos_del'),
    path('alumnos_findEdit/<str:pk>', views.alumnos_findEdit,
         name='alumnos_findEdit'),
    path('alumnosUpdate', views.alumnosUpdate, name='alumnosUpdate'),
]
```

`<str:pk>` recibe el rut del alumno desde la URL. Ejemplo: alumnos_del/1-1

# LISTAR: LA PANTALLA PRINCIPAL DEL CRUD

alumnos/views.py

```python
def crud(request):
    alumnos = Alumno.objects.all()
    return render(request, 'alumnos/alumnos_list.html', {'alumnos': alumnos})
```

templates/alumnos/alumnos_list.html

```
<a href="{% url 'alumnosAdd' %}">Agregar</a>
<div id="idMensajes">{% if mensaje %}{{ mensaje }}{% endif %}</div>
<table>
  {% for x in alumnos %}
  <tr>
    <td>{{ x.rut }}</td> <td>{{ x.nombre }}</td>
    <td><a href="{% url 'alumnos_del' pk=x.rut %}">Eliminar</a></td>
    <td><a href="{% url 'alumnos_findEdit' pk=x.rut %}">Modificar</a></td>
  </tr>
  {% endfor %}
</table>
```

**`pk=x.rut`**
Cada fila arma su propio enlace con el rut. Así Eliminar y Modificar saben sobre qué alumno actuar.



**Probar en:**
`/alumnos/crud`

# AGREGAR: UNA VISTA, DOS MOMENTOS

alumnos/views.py

```python
def alumnosAdd(request):
    generos = Genero.objects.all()
    if request.method != "POST":
        # GET: viene del botón Agregar, se muestra el formulario
        return render(request, 'alumnos/alumnos_add.html', {'generos': generos})

    # POST: viene del formulario, se graban los datos
    objGenero = Genero.objects.get(id_genero=request.POST["genero"])
    Alumno.objects.create(
        rut=request.POST["rut"], nombre=request.POST["nombre"],
        apellido_paterno=request.POST["paterno"],
        apellido_materno=request.POST["materno"],
        fecha_nacimiento=request.POST["fechaNac"], id_genero=objGenero,
        telefono=request.POST["telefono"], email=request.POST["email"],
        direccion=request.POST["direccion"], activo=1)
    context = {'generos': generos, 'mensaje': "Ok, datos grabados..."}
    return render(request, 'alumnos/alumnos_add.html', context)
```

**GET**

**Se pincha Agregar en la lista**
Muestra el formulario vacío con los géneros cargados en el select.

**POST**

**Se envía el formulario**
Lee request.POST, busca el Genero y crea el Alumno en la tabla.

# AGREGAR: EL FORMULARIO HTML

templates/alumnos/alumnos_add.html

```
<form action="{% url 'alumnosAdd' %}" method="POST">
  {% csrf_token %}
  <pre>Rut:     <input type="text" name="rut"></pre>
  <pre>Nombre:  <input type="text" name="nombre"></pre>
  <!-- ... paterno, materno, fechaNac, telefono, email, direccion -->
  <pre>Género:  <select name="genero">
    {% for genero in generos %}
      <option value="{{ genero.id_genero }}">{{ genero.genero }}</option>
    {% endfor %}
  </select></pre>
  <input type="submit" value="Agregar">
</form>
<a href="{% url 'crud' %}">Volver</a>
{% if mensaje %}{{ mensaje }}{% endif %}
```

👉 `{% csrf_token %}` es obligatorio en todo formulario POST.

**name ⇄ request.POST**

name="paterno" se lee en la vista como `request.POST["paterno"]`

Si los nombres no coinciden, la vista no encuentra el dato.

# ELIMINAR UN REGISTRO

alumnos/views.py

```python
def alumnos_del(request, pk):
    try:
        Alumno.objects.get(rut=pk).delete()
        mensaje = "Bien, datos eliminados..."
    except Alumno.DoesNotExist:
        mensaje = "Error, rut no existe..."
    context = {'alumnos': Alumno.objects.all(), 'mensaje': mensaje}
    return render(request, 'alumnos/alumnos_list.html', context)
```



**try / except** evita que la página falle si el rut ya no existe: en ese caso se muestra un mensaje de error.

**1 Enlace**

Eliminar envía el rut: alumnos_del/1-1

**2 Buscar**

get(rut=pk) obtiene el objeto Alumno

**3 Borrar**

delete() lo elimina de la tabla

**4 Volver**

Se muestra la lista con el mensaje

# MODIFICAR (1): BUSCAR Y MOSTRAR

alumnos/views.py

```python
def alumnos_findEdit(request, pk):
    alumno = Alumno.objects.get(rut=pk)
    context = {'alumno': alumno, 'generos': Genero.objects.all()}
    return render(request, 'alumnos/alumnos_edit.html', context)
```

templates/alumnos/alumnos_edit.html

```html
<form action="{% url 'alumnosUpdate' %}" method="POST">
  {% csrf_token %}
  <input type="text" name="rut" value="{{ alumno.rut }}" readonly>
  <input type="text" name="nombre" value="{{ alumno.nombre }}">
  <!-- ... resto de los campos, cada uno con su value -->
  {% for genero in generos %}
    {% if genero.genero == alumno.id_genero.genero %}
      <option value="{{ genero.id_genero }}" selected>{{ genero.genero }}</option>
    {% else %}
      <option value="{{ genero.id_genero }}">{{ genero.genero }}</option>
    {% endif %}
  {% endfor %}
```

**Cambios respecto a Agregar**

- action → alumnosUpdate
- value muestra el dato actual
- selected marca el género del alumno
- rut en readonly

# MODIFICAR (2): GRABAR LOS CAMBIOS

alumnos/views.py

```python
def alumnosUpdate(request):
    alumno = Alumno.objects.get(rut=request.POST["rut"])
    alumno.nombre = request.POST["nombre"]
    alumno.apellido_paterno = request.POST["paterno"]
    alumno.apellido_materno = request.POST["materno"]
    alumno.fecha_nacimiento = request.POST["fechaNac"]
    alumno.id_genero = Genero.objects.get(id_genero=request.POST["genero"])
    alumno.telefono = request.POST["telefono"]
    alumno.email = request.POST["email"]
    alumno.direccion = request.POST["direccion"]
    alumno.save()                          # UPDATE en la tabla
    context = {'alumno': alumno, 'generos': Genero.objects.all(),
               'mensaje': "Ok, datos actualizados..."}
    return render(request, 'alumnos/alumnos_edit.html', context)
```

**`get()`**
obtiene el alumno que ya existe

**`asignar`**
cada atributo recibe el valor del formulario

**`save()`**
como el objeto ya existe, Django hace UPDATE y no INSERT



**¡Listo!** El CRUD de alumnos está completo.

# ¿QUÉ ES UN FORM DE DJANGO?

Forms crea y gestiona un formulario a partir del modelo, con un mínimo de código. En vez de escribir cada `<input>`, se escribe una sola variable.

**20**

líneas con <input> a mano

**4**

**líneas con Forms**

templates/alumnos/generos_add.html

```
<form action="" method="post">
    {% csrf_token %}
    {{ form.as_table }}
    <input type="submit" value="Agregar">
</form>
```

👉 `{{ form.as_table }}` dibuja automáticamente las etiquetas y los campos definidos en el modelo.

# CREAR UN FORM CON MODELFORM

alumnos/forms.py

```
from django.forms import ModelForm
from .models import Genero

class GeneroForm(ModelForm):
    class Meta:
        model = Genero
        fields = ["genero"]        # o "__all__"
        labels = {"genero": "Género"}
```

**La subclase Meta indica**

**`model:`** de qué modelo salen los datos

**`fields:`** qué campos se muestran

**`labels:`** el texto de cada campo. Sin labels, Django usa el nombre del campo: “Genero”, sin tilde.

**Tres formas de mostrar un Form en el template**

**`{{ form.as_table }}`**
cada campo como fila de una tabla

**`{{ form.as_ul }}`**
cada campo como elemento de una lista

**`{{ form.as_p }}`**
cada campo como un párrafo

# CRUD CON FORMS: RUTAS, LISTAR Y ELIMINAR

alumnos/urls.py

```
path('crud_generos', views.crud_generos, name='crud_generos'),
path('generosAdd', views.generosAdd, name='generosAdd'),
path('generos_del/<str:pk>', views.generos_del, name='generos_del'),
path('generos_edit/<str:pk>', views.generos_edit, name='generos_edit'),
```

alumnos/views.py

```
from .forms import GeneroForm

def crud_generos(request):
    generos = Genero.objects.all()
    return render(request, 'alumnos/generos_list.html', {'generos': generos})

def generos_del(request, pk):
    try:
        Genero.objects.get(id_genero=pk).delete()
        mensaje = "Bien, datos eliminados..."
    except Genero.DoesNotExist:
        mensaje = "Error, id no existe..."
    context = {'generos': Genero.objects.all(), 'mensaje': mensaje}
    return render(request, 'alumnos/generos_list.html', context)
```



**Igual que en alumnos.** Listar y eliminar no cambian con Forms; la diferencia está en agregar y modificar.

# AGREGAR CON FORMS

alumnos/views.py

```
def generosAdd(request):
    mensaje = ""
    if request.method == "POST":
        form = GeneroForm(request.POST)   # datos del formulario
        if form.is_valid():               # valida según el modelo
            form.save()                   # INSERT en la tabla
            form = GeneroForm()           # limpia el formulario
            mensaje = "Ok, datos grabados..."
    else:
        form = GeneroForm()               # formulario vacío
    context = {'form': form, 'mensaje': mensaje}
    return render(request, 'alumnos/generos_add.html', context)
```

**`GeneroForm(request.POST)`**
carga en el form los datos enviados

**`form.is_valid()`**
revisa largo, obligatorios y tipos según el modelo

**`form.save()`**
graba el registro, sin leer campo por campo

# MODIFICAR CON FORMS

alumnos/views.py

```python
def generos_edit(request, pk):
    genero = Genero.objects.get(id_genero=pk)
    mensaje = ""
    if request.method == "POST":
        form = GeneroForm(request.POST, instance=genero)
        if form.is_valid():
            form.save()                    # UPDATE del registro
            mensaje = "Bien, datos actualizados..."
    else:
        form = GeneroForm(instance=genero)   # form con datos actuales
    context = {'genero': genero, 'form': form, 'mensaje': mensaje}
    return render(request, 'alumnos/generos_edit.html', context)
```

👉 `instance=genero` es la clave: carga el registro en el formulario y hace que save() actualice en lugar de insertar.

**`generos_edit.html`**

Es igual a generos_add.html: solo cambia el texto del botón a **Actualizar**.

# RESUMEN: CRUD A MANO V/S CRUD CON FORMS

| Operación | CRUD a mano (alumnos) | CRUD con Forms (géneros) |
|---|---|---|
| **Formulario HTML** | Un <input> por campo, ~20 líneas | `{{ form.as_table }}` |
| **Listar** | `Alumno.objects.all()` | Igual |
| **Agregar** | `request.POST[...] campo a campo`<br>`+ Alumno.objects.create(...)` | `GeneroForm(request.POST)`<br>`+ form.save()` |
| **Modificar** | `get() + asignar cada campo`<br>`+ alumno.save()` | `GeneroForm(request.POST, instance=obj)`<br>`+ form.save()` |
| **Eliminar** | `Alumno.objects.get(rut=pk).delete()` | Igual |
| **Validación** | Se programa a mano | `form.is_valid()` |



**Idea central:** con ORM no escribimos SQL; con Forms tampoco escribimos el formulario campo por campo.



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

# Django: ORM, CRUD y Forms

Nancy Bernal Sánchez · Nancy.bernal@usm.cl